



第11章 Pandas数据处理与分析

赵晓航

xiaohangzhao@mail.shufe.edu.cn

上海财经大学·信息管理与工程学院



目录

1. **Pandas概述**
2. **Pandas数据结构**
3. **数据导入与导出**
4. **数据清洗**
5. **数据分析与操作**
6. **案例：清洗与整理员工信息数据集**



1. Pandas概述

- **Pandas :**

- Pandas 是一个开源的 Python 库，提供高效的数据结构和分析工具，核心数据结构包括 **Series（用于一维数据）** 和 **DataFrame（用于二维数据）**
- 借助 Pandas，可以轻松完成数据清洗、合并、重塑、统计分析等操作，从而简化结构化数据的处理过程

- **Pandas的主要应用场景有：**

- **数据清洗：** 处理缺失值、重复值和异常值
- **数据转换：** 格式转换、类型转换和数据重塑
- **数据分析：** 统计分析、聚合操作和时间序列分析
- **数据可视化：** 与其他可视化库（如Matplotlib）结合，快速绘制图表



• 安装与配置Pandas

- 在使用 Pandas 之前，需要先在 Python 环境中安装 Pandas 库
 - `pip install pandas`
- 安装完成后，可以通过以下命令验证 Pandas 是否安装成功并查看版本号：
 - `import pandas as pd`
 - `print(pd.__version__)`
- 需要注意的是，Pandas 依赖于 NumPy 库，因此在安装 Pandas 时会自动安装或更新 NumPy



2. Pandas数据结构

• 2.1 Series数据结构

- Series 是 Pandas 中最基本的数据结构，用于表示一维、带标签的数组。它可以存储多种类型的数据，如整数、浮点数和字符串等，并通过标签（索引）与数据项相关联。相比传统的 Python 数组或列表，Series 提供了更高的灵活性和功能。
- (1) 创建和初始化Series**
- Series 可以通过列表、字典或 NumPy 数组创建，并可自定义索引。如果未指定索引，Pandas 会自动生成从 0 开始的整数索引
- 自定义索引可以使数据的访问和操作更加直观和方便，比如从字典创建的 Series 将自动使用字典的键作为索引
- 1) 从列表创建Series
 - `import pandas as pd`
 - `series1 = pd.Series([1, 2, 3])`
 - `print(series1)`
- 2) 从字典创建Series
 - `data_dict = {'a': 1, 'b': 2, 'c': 3}`
 - `series2 = pd.Series(data_dict)`
 - `print(series2)`
- 3) 从NumPy数组创建Series
 - `import numpy as np`
 - `array_data = np.array([10, 20, 30])`
 - `series3 = pd.Series(array_data)`
 - `print(series3)`



(2) Series的基本操作与属性

- Series 提供多种属性和方法，用于数据操作和信息获取。
 - 1) 查看数据类型：**使用 dtype 属性查看 Series 中数据的类型。
 - `series4 = pd.Series([5,6,7,8,9,10])`
 - `print(series4.dtype)` # 输出 int64
 - 2) 获取统计信息：**使用 describe() 方法快速查看数值数据的统计信息。
 - `print(series4.describe())`
 - 3) 关键属性：**
 - Series对象的values属性返回Series中的数据部分（不包括索引）
 - index属性返回Series的索引
 - 而name属性则是Series的名称，用于标识该Series。

```
count    6.000000
mean     7.500000
std      1.870829
min      5.000000
25%      6.250000
50%      7.500000
75%      8.750000
max     10.000000
dtype: float64
```



(3) Series索引和切片

- Series 支持灵活的索引和切片功能，包括位置索引、标签索引、切片和布尔索引。
- 1) **位置索引**：使用整数位置访问数据。
 - `print(series4[0])` # 输出第一个元素
- 2) **标签索引**：使用标签访问与标签对应的数据。
 - `print(series2['a'])` # 输出标签为 'a' 的元素的值，如1
- 3) **切片操作**：切片操作允许选择 Series 中的一部分数据，与 Python 列表的切片操作类似。通过指定起始和结束位置，可以获取 Series 的一个子集，但不包括结束位置的元素。
 - `print(series4[1:4])` # 输出从第1个到第3个（不包含第4个）的元素
 - 当使用标签切片时，Pandas 的行为不同于 Python 列表。**标签切片会包含终止标签对应的元素。**
 - `print(series2['a':'c'])` # 输出从标签 'a' 到标签 'c'（包含 'c'）的所有元素
- 4) **布尔索引**：使用布尔索引来筛选符合特定条件的元素。布尔索引是通过布尔表达式生成的一个与Series长度相同的布尔值序列（True或False）。根据这些布尔值来筛选出对应位置为True的元素。
 - `print(series4[series4 > 3])` # 输出大于3的元素



2.2 DataFrame数据结构

- **DataFrame:** 用于表示二维、带标签的数据表，类似于 Excel 表格或 SQL 表格。DataFrame 每列可以存储不同类型的数据，并支持灵活的行列操作，使其成为处理复杂数据集的首选工具
- 1. 创建和初始化DataFrame
 - DataFrame 可以通过多种方式创建，包括从字典、嵌套列表、NumPy 数组等数据结构。
 - 每个 DataFrame 的行和列都有各自的索引，默认情况下 Pandas 为其生成整数索引，也可通过参数自定义索引
 - 例13.1 利用字典创建一个DataFrame。
 - `import pandas as pd`
 - `data_dict = {`
 - `'姓名': ['张伟', '王芳', '李娜'],`
 - `'年龄': [25, 30, 35],`
 - `'城市': ['北京', '上海', '广州']`
 - `}`
 - `df = pd.DataFrame(data_dict)`
 - `print(df)`

运行结果：

	姓名	年龄	城市
0	张伟	25	北京
1	王芳	30	上海
2	李娜	35	广州
 - 还可以从嵌套列表创建DataFrame，并为其指定列名：
 - `data_list = [['张伟', 25, '北京'],`
 - `['王芳', 30, '上海'],`
 - `['李娜', 35, '广州']]`
 - `df2 = pd.DataFrame(data_list, columns=['姓名', '年龄', '城市'])`
 - `print(df2)`
 - DataFrame中的数据可以是混合类型的，不同列可以包含不同类型的数据，如字符串、整数、浮点数等。DataFrame的列标签使得数据操作更加直观，并且可以通过标签方便访问和修改数据。



• 2. DataFrame的基本属性与操作

- DataFrame支持多种常见的属性和操作，如数据选择、统计分析、数据重命名等
- (1) **查看 DataFrame 的维度**：使用 shape 属性查看 DataFrame 的行数和列数。
 - `print(df.shape)` # 输出 (3, 3), 表示 3 行 3 列
- (2) **columns 属性**：返回 DataFrame 的列标签。
 - `print(df.columns)` # 输出列名列表
- (3) **index 属性**：返回 DataFrame 的行索引。
 - `print(df.index)` # 输出行索引
- (4) **dtypes 属性**：返回每列的数据类型。
 - `print(df.dtypes)` # 输出列的数据类型
- (5) **查看前几行数据**：使用 head() 方法查看前几行数据，默认输出前 5 行，可通过参数设置具体行数。
 - `print(df.head(2))` # 查看前 2 行
- (6) **获取数值列的统计信息**：使用 describe() 方法生成数值列的描述性统计信息，包括计数、均值、标准差、最小值、最大值及分位数等：
 - `print(df.describe())` # 输出描述性统计信息



• 3. DataFrame索引和切片

- 与 Series 类似，DataFrame 也支持基于位置或标签的索引和切片。Pandas 提供了多种灵活的索引机制
- 1) **列标签**：可以直接使用列标签访问 DataFrame 中的某一列。
 - `print(df['姓名'])` # 输出姓名列的数据
 - 运行结果：
 - 0 张伟
 - 1 王芳
 - 2 李娜
 - Name: 姓名, dtype: object
- 2) **loc()方法**：loc() 方法通过行标签和列标签进行索引，适用于基于标签的精确数据访问。
 - `print(df.loc[0, '姓名'])` # 输出第一行“姓名”列的数据，即张伟
- 3) **iloc()方法**：iloc() 方法通过整数位置进行索引，与传统 Python 列表操作类似。
 - `print(df.iloc[0, 1])` # 输出第1行第2列的数据，即25
 - 切片操作允许选择DataFrame中的一个子集，例如：
 - `print(df.iloc[1:3, :])` # 输出第2行到第3行的所有列
 - 运行结果：
 - 姓名 年龄 城市
 - 1 王芳 30 上海
 - 2 李娜 35 广州



• 4. DataFrame的行列操作

- DataFrame支持对行列的灵活操作，包括添加、删除和修改行列的内容。
- 1) **添加新行**：使用`_append()`方法或者`pd.concat`函数添加新的行。通过创建一个 Series 对象并将其追加到 DataFrame 中实现。
 - `new_row = pd.Series({'姓名': '赵磊', '年龄': 40, '城市': '深圳'})`
 - `df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)`
 - `print(df)`
 - 其中参数`ignore_index=True`表示在追加新行后重新生成索引，而不使用新行原有的索引。
- 2) **添加新列**：通过赋值语句添加一列数据，例如，以下代码添加一列表示性别的数据：
 - `df['性别'] = ['男', '女', '女']`
- 3) **删除行**：使用`drop()`方法删除指定行的索引来删除一行数据。
 - `df.drop(index=0, inplace=True) # 删除第一行`
 - 在这里，参数`index=0`表示删除第一行。`inplace=True`用于指定在原 DataFrame 上直接删除行，不返回新对象；如果设为 `False`，会返回一个删除指定行的新 DataFrame，原对象保持不变。
- 4) **删除列**：使用`drop()`方法删除指定列名的一列数据。
 - `df.drop(columns=['年龄'], inplace=True) # 删除Age列`



3. 数据导入与导出

- Pandas支持从多种格式的文件或数据库中读取数据到 DataFrame，并将分析结果导出到外部文件。

- **1. 读取数据文件**

- Pandas 支持读取多种格式的数据文件，包括 CSV、Excel、JSON，以及 SQL 数据库，可以容易地从外部数据源获取数据，并将其加载到Pandas的DataFrame中进行处理和分析。

- (1) 读取CSV文件

- 利用read_csv()函数，方便地将CSV文件读入DataFrame中。

- `import pandas as pd`

- `# 读取CSV文件`

- `df = pd.read_csv('GDP.csv')`

- `print(df.head(3))`

- 运行结果：

- | | 指标 | 2017年 | 2018年 | 2019年 | 2020年 |
|---|-------------|----------|----------|----------|-----------|
| 0 | 国民总收入(亿元) | 830945.7 | 915243.5 | 983751.2 | 1008782.5 |
| 1 | 国内生产总值(亿元) | 832035.9 | 919281.1 | 986515.2 | 1015986.2 |
| 2 | 第一产业增加值(亿元) | 62099.5 | 64745.2 | 70473.6 | 77754.1 |

- 在此例中，pd.read_csv('GDP.csv')将文件GDP.csv读取为一个DataFrame对象。head(3)方法用于查看前3行数据，以验证数据加载是否正确。read_csv()函数还支持多种参数，如指定分隔符sep、缺失值处理na_values、列类型转换dtype等，以应对不同格式和内容的CSV文件。

- (2) **读取Excel文件:** 利用read_excel()函数, 可以直接读取Excel文件中的数据。
 - `df = pd.read_excel('data.xlsx', sheet_name='Sheet1')`
 - `print(df.head())`
 - 在此例中, pd.read_excel('data.xlsx', sheet_name='Sheet1')从指定的Excel文件中的Sheet1工作表读取数据, sheet_name参数用于指定要读取的工作表名称或索引, 若省略该参数, 则默认读取第一个工作表。
- (3) **读取JSON :** Pandas 提供了 read_json() 函数来处理 JSON 格式的数据。
 - `df = pd.read_json('data.json')`
 - `print(df.head())`
 - 这里pd.read_json('data.json')方法将JSON文件转换为DataFrame对象, 支持处理复杂的嵌套数据结构。
- (4) **读取SQL数据库:** SQLite是一种轻量级的嵌入式数据库, 它是独立的、无服务器的、自包含的, 并且不需要安装任何其他的数据库管理系统。Pandas内置对SQLite数据库的支持, 可以轻松地将SQL查询结果直接加载到DataFrame中。
 - 例13.2 从SQLite数据库读取数据
 - `import sqlite3`
 - `import pandas as pd`
 - `# 创建或连接到SQLite数据库`
 - `conn = sqlite3.connect('database.db')`
 - `# 使用SQL查询从数据库读取数据到DataFrame`
 - `df = pd.read_sql('SELECT * FROM tablename', conn)`
 - `print(df.head())`
 - `# 关闭数据库连接`
 - `conn.close()`

• 2. 写入数据文件

- 在完成数据分析后，通常需要将结果保存为文件以便共享或存档。Pandas 提供了多种导出数据的功能，支持将 DataFrame 导出为多种格式，如 CSV、Excel、JSON 和 SQL 数据库。
- (1) **导出CSV文件**: 使用 `to_csv()` 方法可以将 DataFrame 导出为 CSV 文件。
 - `df.to_csv('output.csv', index=False)`
 - 这里`df.to_csv('output.csv', index=False)`将DataFrame导出为名为output.csv的文件。`index=False`参数确保导出时不包含行索引，这通常简化了文件结构，便于与其他系统集成。
- (2) **导出Excel文件**: 使用 `to_excel()` 方法可以将 DataFrame 导出为 Excel 文件。
 - `df.to_excel('output.xlsx', sheet_name='Sheet1', index=False)`
 - 这里`df.to_excel('output.xlsx', sheet_name='Sheet1', index=False)`将DataFrame导出为output.xlsx文件，并将数据存储在Sheet1工作表中，`sheet_name`参数指定工作表的名称，`index=False`参数则省略行索引。
- (3) **导出JSON 文件**: 使用 `to_json()` 方法可以将 DataFrame 导出为 JSON 格式的数据文件。
 - `df.to_json('output.json')`
- (4) **导出到 SQL 数据库**: Pandas 提供了将数据直接存储到 SQL 数据库的能力，适合与数据库系统的集成。
 - `import sqlite3`
 - `# 创建或连接到SQLite数据库`
 - `conn = sqlite3.connect('database.db')`
 - `# 导出为SQL数据库中的表`
 - `df.to_sql('tablename', conn, if_exists='replace', index=False)`
 - `# 关闭数据库连接`
 - `conn.close()`



4. 数据清洗

- 原始数据中经常包含缺失值、冗余数据或格式不统一的问题，这些问题可能导致分析结果的误导性。
- Pandas 提供了丰富工具帮助完成数据清洗任务，常见操作包括处理缺失数据、数据过滤与选择，以及数据转换。

(1) 缺失数据处理

- 缺失数据是数据清洗中最常见的问题之一。Pandas 提供了一系列方法来检测、填充或删除缺失数据。
- 1. 检测缺失数据 `isna()` 方法
 - 在清洗数据之前，首先需要了解哪些数据缺失。Pandas 提供了 `isna()` 和 `notna()` 方法，用于检测 DataFrame 中的缺失数据。
 - `import pandas as pd`
 - `# 创建一个包含缺失值的 DataFrame`
 - `data = {'姓名': ['张伟', '李娜', '王芳', None],`
 - 为了更直观地了解缺失数据的总体情况，可以统计每一列中缺失值的数量。
 - `'年龄': [25, None, 30, 35],`
 - `'城市': ['北京', '上海', None, '广州']}`
 - `df = pd.DataFrame(data)`
 - `# 统计每列的缺失值数量`
 - `# 检测缺失值`
 - `print(df.isna())`
 - `print(df.isna().sum())`
 - 运行结果：
 - 姓名 1
 - 年龄 1
 - 城市 1
 - `dtype: int64`
 - 运行结果：
 - | | 姓名 | 年龄 | 城市 |
|---|-------|-------|-------|
| 0 | False | False | False |
| 1 | False | True | False |
| 2 | False | False | True |
| 3 | True | False | False |
 - `df.isna()` 返回一个与原 DataFrame 形状相同的布尔值 DataFrame，其中 True 表示该位置存在缺失值，False 表示数据完整。



2. 填充缺失数据fillna() 方法

- 在定位到缺失值后，可以使用 fillna() 方法填充数据。

(1) 用平均值填充数值列的缺失值

- # 用平均值填充缺失的年龄数据
- `df['年龄'] = df['年龄'].fillna(value=df['年龄'].mean())`

(2) 前向填充 (ffill) 和后向填充 (bfill)

- 前向填充**：将缺失值替换为前一个非缺失值，适用于时间序列数据。
- # 前向填充缺失的城市数据
- `df['城市'] = df['城市'].fillna(method='ffill')`
- 后向填充**：将缺失值替换为后一个非缺失值。
- # 后向填充缺失的城市数据
- `df['城市'] = df['城市'].fillna(method='bfill' )`

(3) 删除缺失数据dropna()方法

- 如果填充缺失值不合适，可以选择删除包含缺失数据的行或列。

(1) 删除包含缺失值的行

- # 删除包含缺失数据的行
- `df_dropped = df.dropna()`

- 代码中dropna() 方法默认删除所有包含缺失数据的行。

(2) 删除包含缺失值的列。

- # 删除包含缺失数据的列
- `df_dropped_cols = df.dropna(axis=1)`
- 代码中dropna() 方法的axis=1 参数指定删除列而非行。



- **(2) 数据过滤与选择**

- 数据过滤与选择可帮助从庞大的数据集中提取出符合特定条件的子集，进行更有针对性的分析。
- **1. 布尔索引和条件过滤**
 - 使用布尔索引和条件表达式，可以快速筛选出满足特定条件的数据行。
 - # 筛选出年龄大于30的人员
 - `filtered_df = df[df['年龄'] > 30]`
 - 在该示例中，布尔表达式 `df['Age'] > 30` 返回一个布尔 Series，Pandas 使用这个布尔 Series 对原 DataFrame 进行过滤操作，返回符合条件的行。
- **2. 多条件过滤**
 - Pandas 支持使用逻辑运算符组合多个条件，以实现多条件筛选。常见的逻辑运算符包括 &（与）、|（或）和 ~（非）。
 - # 筛选出年龄大于 30 且居住在 '北京' 的人员
 - `filtered_df = df[(df['年龄'] > 30) & (df['城市'] == '北京')]`
 - 在该例中，使用圆括号对每个条件进行分组，确保逻辑运算符的优先级正确；逻辑运算符 & 生成一个新的布尔 Series，筛选出同时满足两个条件的行。



3. 使用query()方法进行数据筛选

- query() 是一个简洁的查询方法，用于基于表达式筛选 DataFrame 中的数据。与布尔索引相比，query() 使用字符串表达式指定条件，语法更接近 SQL 风格，代码可读性更高。
- query()的语法: `DataFrame.query(expr, inplace=False, **kwargs)`
- 其中expr为字符串格式的查询表达式，指定筛选条件；inplace表示是否直接在原 DataFrame 上修改数据，默认 False；kwargs 表示查询表达式中需要用到外部变量。

(1) 按条件筛选数据

- 假设有一个包含学生成绩的 DataFrame，想筛选数学成绩大于 80 且英语成绩大于 75 的学生。
- `import pandas as pd`
- `# 创建数据集`
- `data = {'姓名': ['张伟', '李娜', '王芳', '赵磊'],`
`'数学': [85, 78, 92, 88],`
`'英语': [80, 72, 95, 89]}`
- `df = pd.DataFrame(data)`
- `# 使用 query() 筛选数据`
- `result = df.query("数学 > 80 and 英语 > 75")`
- `print(result)`
- 在df.query()方法中，条件“数学 > 80 and 英语 > 75”表示筛选出数学成绩大于 80 且英语成绩大于 75 的学生，且查询语法直接使用列名，无需引号或其他标记。

运行结果:

	姓名	数学	英语
0	张伟	85	80
2	王芳	92	95
3	赵磊	88	89

(2) 使用动态变量

- 当筛选条件需要依赖外部变量时，可以使用 @ 引入变量。
- `# 定义动态变量`
- `math_threshold = 85`
- `# 使用动态变量筛选数据`
- `result = df.query("数学 > @math_threshold")`
- `print(result)`
- 运行结果:

	姓名	数学	英语
2	王芳	92	95
3	赵磊	88	89

(3) 基于字符串筛选数据

- query() 还支持字符串操作，例如筛选姓名包含“张”的学生。
- `result = df.query("姓名.str.contains('张')", engine='python')`
- `print(result)`
- 在df.query()方法中，参数engine的默认引擎是 numexpr，支持快速的数值计算，但不支持复杂的字符串操作。如需字符串操作，需将 engine 设置为 python。
- 运行结果:

	姓名	数学	英语
0	张伟	85	80



(3) 数据转换：将数据从一种形式转换为另一种形式

1. 数据类型转换`astype()`方法

- 使用`astype()`方法可以转换数据类型。
- # 将年龄列转换为整数类型
- `df['年龄'] = df['年龄'].astype(int)`
- `astype(int)` 将 年龄 列的数据类型从浮点数转换为整数。通过 `dtypes` 属性可以检查 `DataFrame` 中每一列的数据类型，以验证转换是否成功。

2. 删除重复数据`drop_duplicates()` 方法

- 数据集中重复的数据可能会影响分析的精确性，Pandas 提供了 `drop_duplicates()` 方法快速删除重复行。
- `df_unique = df.drop_duplicates()`
- 其中`drop_duplicates()` 默认保留每组重复行中的第一行，删除其他重复行。
- 可以通过参数调整行为，例如保留最后一行。
- `df_unique = df.drop_duplicates(keep='last')`
- 也可以根据特定列判断重复行：
- `df_unique = df.drop_duplicates(subset=['列名'])`

3. 字符串操作与处理`str`

- 数据中常包含大量字符串数据，Pandas 提供了丰富的字符串操作方法，通过 `str` 属性可以完成常见任务，例如大小写转换、去除空白、替换字符等。
- # 去除字符串两端的空白字符
- `df['城市'] = df['城市'].str.strip()`
- # 替换特定字符或子串
- `df['城市'] = df['城市'].str.replace('北京', 'Beijing')`
- 在该例中，`str.strip()` 方法去除字符串两端的空白，而`str.replace('北京', 'Beijing')` 方法查找原始字符串“北京”，以替换为目标字符串“Beijing”。



5. 数据分析与操作

- 数据分析主要包括数据排序、索引管理、数据分组与聚合，以及数据合并与重塑等操作

1. 数据排序与索引

- 数据排序与索引是数据处理的基础操作。排序可以使数据按特定顺序查看，而索引则提供了高效的数据访问和管理
- **1. 排序操作：**两种常用的排序方法：
 - (1) **根据列值排序sort_values()方法**
 - `import pandas as pd`
 - `data = {'姓名': ['张伟', '李娜', '王芳', '赵磊'],`
 - `'年龄': [25, 30, 35, 40],`
 - `'城市': ['北京', '上海', '广州', '深圳']}`
 - `df = pd.DataFrame(data)`
 - `df_sorted = df.sort_values(by='年龄')`
 - `print(df_sorted)`
 - 在该例中，`df.sort_values(by='年龄')`根据“年龄”列的值对DataFrame进行升序排序，并设置`ascending=False`参数实现降序排序。
 - **多列排序：**可以同时根据多个列排序，优先级按照给出的列名顺序。
 - # 先按城市列排序，再按年龄列排序
 - `df_sorted = df.sort_values(by=['城市', '年龄'])`
 - (2) **根据索引排序sort_index()方法**
 - `sort_index()`用于按DataFrame或Series的行索引或列索引（`axis=1`）进行排序。它可以选择按升序或降序排列，也可以指定按行或列进行排序。
 - # 按列索引排序
 - `df_sorted_by_columns = df.sort_index(axis=1)`
- 运行结果：

	姓名	年龄	城市
0	张伟	25	北京
1	李娜	30	上海
2	王芳	35	广州
3	赵磊	40	深圳



• 2. 设置和重置索引

- 索引是高效管理和访问数据的关键部分。可以通过 `set_index()` 方法设置新的索引，也可以通过 `reset_index()` 恢复为默认整数索引。

- (1) 设置索引|`set_index()`方法

- # 将姓名列设置为索引
- `df_indexed = df.set_index('姓名')`
- `print(df_indexed)`
- 在此示例中，`df.set_index('姓名')` 将“姓名”列设为索引，以便快速通过名称访问数据。原索引列从 DataFrame 中移除。
- 运行结果：
- 年龄 城市
- 姓名
- 张伟 25 北京
- 李娜 30 上海
- 王芳 35 广州
- 赵磊 40 深圳

- (2) 重置索引|`reset_index()`方法

- # 重置索引

- `df_reset = df_indexed.reset_index()`
- `print(df_reset)`
- 其中`reset_index()` 方法将索引重置为默认的整数索引，并将原来的索引列恢复为普通列。

- 运行结果：

- 姓名 年龄 城市
- 0 张伟 25 北京
- 1 李娜 30 上海
- 2 王芳 35 广州
- 3 赵磊 40 深圳



• 2. 数据分组与聚合

- 数据聚合与分组是数据分析中的核心任务之一，特别是在对不同类别的数据进行汇总和统计时非常有用。

(1) 分组操作groupby() 方法

- 分组操作允许根据一个或多个列的值将数据分为不同的组，然后对每个组分别进行计算。Pandas 的 groupby() 方法可以实现数据分组。
- # 创建一个包含更多数据的 DataFrame
- import pandas as pd
- data = {'姓名': ['张伟', '李娜', '王芳', '赵磊', '陈刚', '刘敏'],
- '年龄': [28, 30, 32, 40, 25, 35],
- '城市': ['北京', '上海', '广州', '深圳', '北京', '广州']}
- df = pd.DataFrame(data)
- # 根据城市列进行分组
- grouped = df.groupby('城市')
- 在此示例中，df.groupby('城市') 根据“城市”列的值将数据分组，返回一个分组对象 (DataFrameGroupBy 对象)，而不是一个普通的 DataFrame。该分组对象可用于下一步计算每个组的统计量。



(2) 聚合函数与分析

- 完成数据分组后，通常需要对每个组进行聚合操作，如求和、求平均值或计数。Pandas 提供了多种聚合函数，包括 `sum()`、`mean()`、`count()` 等。

计算每个城市的人员数量

计算每个城市的平均年龄

- # 接上例，计算每个城市的平均年龄，并更改列名为 '平均年龄'

- `mean_age = grouped['年龄'].mean().rename('平均年龄')`

- `print(mean_age)`

- 其中，`mean()` 方法对每个分组的“年龄”列计算平均值，并将该列改名为“平均年龄”。

- 运行结果:

- 城市

- 上海 30.0

- 北京 26.5

- 广州 33.5

- 深圳 40.0

- Name: 平均年龄, dtype: float64

- # 接上例，计算每个城市的人员数量，并更改列名为 '人数'

- `count_people = grouped['姓名'].count().rename('人数')`

- `print(count_people)`

- 其中，`count()` 方法对每个分组的“姓名”列统计非缺失值的数量，并将该列改名为“人数”。

- 运行结果:

- 城市

- 上海 1

- 北京 2

- 广州 2

- 深圳 1

- Name: 人数, dtype: int64



姓名	年龄	城市
张伟	28	北京
李娜	30	上海
王芳	32	广州
赵磊	40	深圳
陈刚	25	北京
刘敏	35	广州

城市	姓名	年龄
北京	张伟	28
	陈刚	25
上海	李娜	30
广州	王芳	32
	刘敏	35
深圳	赵磊	40

城市	年龄
北京	26.5
上海	30.0
广州	33.5
深圳	40.0

城市	人数
北京	2
上海	1
广州	2
深圳	1



agg方法

agg 方法：对DataFrame的列进行聚合操作

===== 1: 准备模拟销售数据 =====

构造包含门店、日期、销量、单价的销售数据表（新手易理解的业务场景）

```
sales_data = {  
    "门店": ["A店", "A店", "B店", "B店", "A店", "B店", "C店", "C店"],  
    "日期": ["2026-01-01", "2026-01-02", "2026-01-01", "2026-01-02",  
            "2026-01-03", "2026-01-03", "2026-01-01", "2026-01-02"],  
    "销量": [50, 65, 30, 45, 70, 55, 20, 35],  
    "单价": [10.5, 10.5, 12.0, 12.0, 10.5, 12.0, 15.0, 15.0]  
}
```

```
df = pd.DataFrame(sales_data)
```

===== 1: 多函数聚合 =====

对指定列（销量）同时执行多个聚合函数（求和、均值、最大值、最小值）

方式1：列表形式指定多函数

```
agg_multi_1 = df["销量"].agg(["sum", "mean", "max", "min"])
```

方式2：对多列执行多函数（销量、单价都执行sum和mean）

```
agg_multi_2 = df[["销量", "单价"]].agg(["sum", "mean"])
```

===== 2. 按列指定不同聚合函数 =====

核心场景：不同列需要不同的聚合逻辑（比如销量求和，单价求均值和最大值）

用字典指定：key=列名，value=聚合函数（单个/多个）

```
agg_col_spec = df.agg({  
    "销量": "sum",      # 销量列：求和  
    "单价": ["mean", "max"] # 单价列：求均值+最大值  
})
```

===== 3. 结合分组 + 按列指定聚合 =====

实战高频场景：先按门店分组，再对每组的销量/单价执行不同聚合

```
group_agg = df.groupby("门店").agg({  
    "销量": ["sum", "mean"], # 每组销量：求和+均值  
    "单价": "max"           # 每组单价：求最大值  
})
```

===== 4. 自定义聚合函数 =====

场景：内置函数满足不了需求时，自定义聚合函数（比如求“极差=最大值-最小值”）

自定义函数1：求极差

```
def range_func(x):  
    """计算序列的极差（max - min）"""  
    return x.max() - x.min()
```

自定义函数2：求中位数绝对偏差（MAD）

```
def mad_func(x):  
    """计算中位数绝对偏差（可理解为“数据离散程度”的指标）"""  
    median = x.median()  
    return (x - median).abs().median()
```

应用自定义函数

```
agg_custom = df.agg({  
    "销量": [range_func, mad_func],  
    "单价": range_func  
})
```



• 3. 数据合并

- 在数据处理过程中，可能需要将多个数据集合并在一起，或者调整数据结构以满足分析需求。Pandas 提供了多种方法，主要包括基于键的合并（merge()、join()方法）和直接拼接（concat() 方法）。

- 1. 基于键的合并merge() 方法**

- merge()方法用于基于一个或多个键，将两个 DataFrame 按列横向合并。它类似于数据库中的 JOIN 操作，并允许根据匹配键选择连接方式，例如内连接、外连接、左连接或右连接。

- `import pandas as pd`

- `# 创建两个 DataFrame`

- `df1 = pd.DataFrame({'姓名': ['张伟', '李娜', '王芳'],`

- `'年龄': [25, 30, 35]})`

- `df2 = pd.DataFrame({'姓名': ['张伟', '李娜', '赵磊'],`

- `'城市': ['北京', '上海', '深圳']})`

- `# 按姓名列进行内连接`

- `merged_df = pd.merge(df1, df2, on='姓名', how='inner')`

- `print(merged_df)`

- 运行结果:

- `姓名 年龄 城市`

- `0 张伟 25 北京`

- `1 李娜 30 上海`

- 在此例中，`pd.merge(df1, df2, on='姓名', how='inner')` 将两个 DataFrame 按 “姓名” 列进行合并，其中 `how='inner'` 表示使用内连接，即只保留在两个 DataFrame 中都存在的 “姓名” 值的记录。



• 2. 基于索引的合并 join() 方法

◦ join() 方法用于**基于索引**将两个 DataFrame 横向合并，适合索引已经对齐的数据集。与 merge() 不同，join() 默认使用索引进行匹配，省去了显式指定键的步骤。此外，join() 也支持内连接、外连接、左连接或右连接等多种连接方式，类似于 SQL 语言中的 JOIN 操作。

```
◦ import pandas as pd
◦ # 创建两个 DataFrame
◦ df1 = pd.DataFrame({'年龄': [25, 30, 35]}, index=['张伟', '李娜', '王芳'])
◦ df2 = pd.DataFrame({'城市': ['北京', '上海', '深圳']}, index=['张伟', '李娜', '赵磊'])
◦ # 按索引进行左连接
◦ joined_df = df1.join(df2, how='left')
◦ print(joined_df)
```

◦ 在该例中，join()方法将 df1 和 df2 按索引匹配，生成一个新的 DataFrame，其中参数how='left' 指定连接方式，表示以左侧的 DataFrame (df1) 为主，保留 df1 的所有索引，并将未匹配的索引值填充为 NaN。如果需要保留右侧所有索引，可以使用 how='outer'。

◦ 运行结果：

```
◦      年龄  城市
◦ 张伟  25.0  北京
◦ 李娜  30.0  上海
◦ 王芳  35.0  NaN
```



• 3. 直接拼接concat() 方法

- concat() 方法用于直接拼接多个 DataFrame，可以按行（垂直拼接）或按列（水平拼接）堆叠数据，不要求键的匹配。

- **(1) 按行拼接**：设置 axis=0 表示按行拼接，生成一个包含所有行的新 DataFrame。

- # 创建两个 DataFrame

- df3 = pd.DataFrame({'姓名': ['张伟', '李娜'],

- '年龄': [25, 30]})

- df4 = pd.DataFrame({'姓名': ['王芳'],

- '年龄': [35]})

- # 纵向合并两个 DataFrame

- concatenated_df = pd.concat([df3, df4], axis=0)

- print(concatenated_df)

- 运行结果:

- 姓名 年龄

- 0 张伟 25

- 1 李娜 30

- 0 王芳 35

- 在此例中，pd.concat([df3, df4], axis=0) 中axis=0 表示按行拼接。将两个 DataFrame 按行进行合并，生成了一个包含 df3 和 df4 的所有行的新 DataFrame。拼接后的行索引默认保留原索引。如果希望重新生成索引，可使用 ignore_index=True 参数。例如，

- concatenated_df = pd.concat([df3, df4], axis=0, ignore_index=True)

- **(2) 按列拼接**：设置 axis=1 表示按列拼接，生成一个包含所有列的新 DataFrame。

- # 创建两个 DataFrame

- df5 = pd.DataFrame({'姓名': ['张伟', '李娜']})

- df6 = pd.DataFrame({'城市': ['北京', '上海']})

- # 按列合并

- concatenated_df_axis1 = pd.concat([df5, df6], axis=1)

- print(concatenated_df_axis1)

- 运行结果:

- 姓名 城市

- 0 张伟 北京

- 1 李娜 上海

- 在此例中，pd.concat([df5, df6], axis=1) 中的axis=1 表示按列拼接，生成了一个包含df5和 df6 列的新 DataFrame。这种方法会直接并排显示所有列，并不需要有共同的键。



• 4. 数据重塑

- 数据重塑 (reshaping) 是将数据从一种布局或格式转换为另一种，以便更有效地分析、展示或存储。Pandas 提供了强大的数据重塑工具，如 `pivot_table()` 和 `melt()`，可以灵活调整 DataFrame 的形状，满足不同的分析需求。
- **1. 数据透视表 `pivot_table()` 方法**
- 数据透视表是一种常见的数据重塑方式，可按指定维度对数据重新组织和汇总。通过 `pivot_table()` 方法，可以根据一个或多个键（通常是列）对数据分组，并计算分组数据的统计量（如总和、平均值等）。这种方法特别适用于多维数据的分析，使复杂的汇总信息清晰直观。
- `pivot` 本身有“旋转、调转”的意思，所以 `pivot_table`（透视表）可以很形象地理解为：把原本表格里**纵向排列**的某一列数据，“旋转调转”成表格里**横向的列索引**，让数据的展示形式更贴合我们的查看和分析需求。



• 例 使用数据透视表汇总中国城市 and 商品类别的销售数据

```
import pandas as pd

# 创建一个包含中国城市、商品类别和销售额的 DataFrame, 增加新记录

data = {
    '城市': ['北京', '北京', '上海', '上海', '广州', '广州', '深圳', '深圳', '北京', '上海'],
    '商品类别': ['电子产品', '家具', '电子产品', '家具', '电子产品', '家具', '电子产品', '家具', '家具', '电子产品'],
    '销售额': [500, 300, 700, 450, 400, 320, 600, 410, 200, 300] # 新增的销售额分别是 200 和 300
}

df = pd.DataFrame(data)

print('原表格: ')

print(df)

print()

# 创建透视表, 按城市 and 商品类别汇总销售额

pivot_table = df.pivot_table(values='销售额', index='城市', columns='商品类别', aggfunc='sum')

print('透视表: ')

print(pivot_table)
```

运行结果:

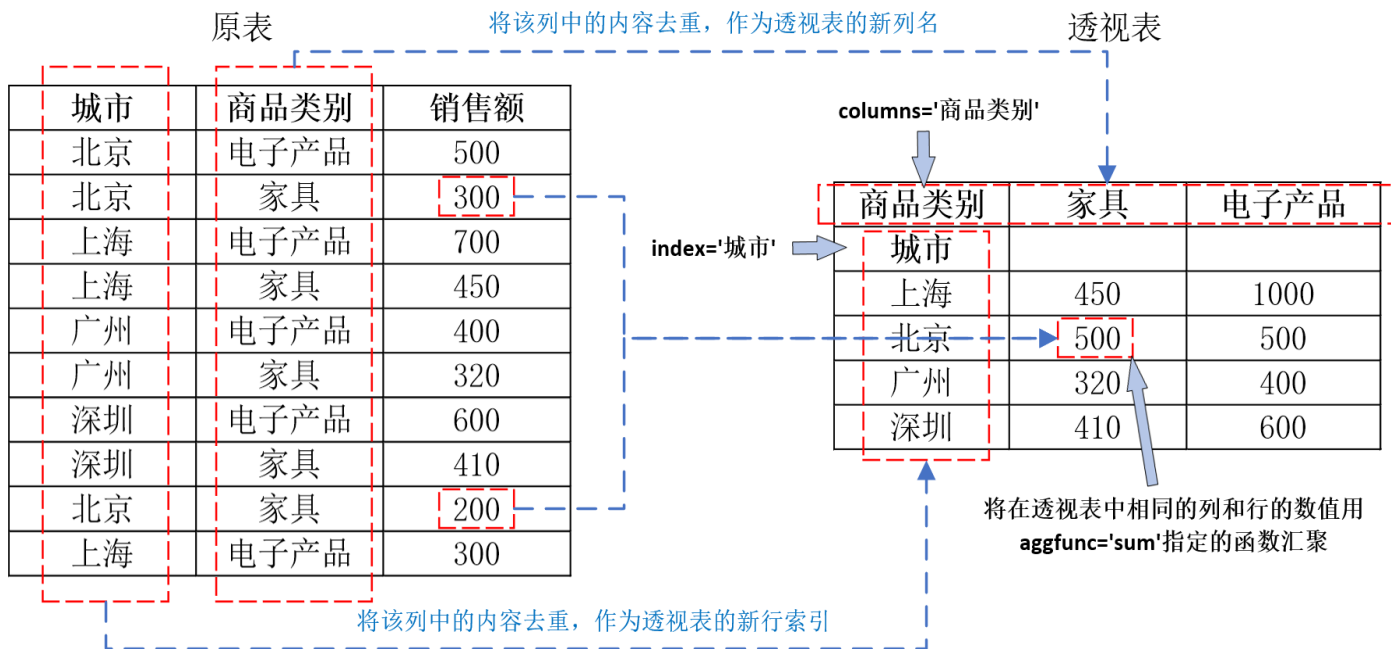
原表格:

	城市	商品类别	销售额
0	北京	电子产品	500
1	北京	家具	300
2	上海	电子产品	700
3	上海	家具	450
4	广州	电子产品	400
5	广州	家具	320
6	深圳	电子产品	600
7	深圳	家具	410
8	北京	家具	200
9	上海	电子产品	300

透视表:

商品类别	家具	电子产品
城市		
上海	450	1000
北京	500	500
广州	320	400
深圳	410	600

- 在此示例中，使用`pivot_table(values='销售额', index='城市', columns='商品类别', aggfunc='sum')`方法，其中参数`values='销售额'`指定需要聚合的列为“销售额”，`index='城市'`指定行索引为“城市”，`columns='商品类别'`指定列索引为“商品类别”，`aggfunc='sum'`表示汇总方法为求和。通过这种数据透视操作，可以快速查看不同城市中不同商品类别的销售总额。该例中的透视表数据重塑过程如图所示。





从词频到词袋向量

letter_doc_df

	doc_id	letter	count
0	1	d	5
1	1	a	1
2	1	x	5
3	1	i	4
4	1	h	1
...	...	...	...
569	100	x	4
570	100	y	1
571	100	w	5
572	100	e	3
573	100	d	2

574 rows × 3 columns

```
bow_vector = letter_doc_df.pivot_table(  
    index="doc_id",      # 行: 文档ID (每个文档对应一行向量)  
    columns="letter",    # 列: 26个英文字母 (纵向转横向)  
    values="count",      # 单元格值: 字母在文档中的出现次数  
    fill_value=0,        # 未出现的字母填充0 (词袋向量核心要求)  
    dropna=False        # 强制保留全部26个字母列 (即使某字母未出现)  
) .reindex(columns=alphabet) # 确保列的顺序是a-z (随机生成后列可能乱序, 整理更易读)
```

letter	a	b	c	d	e	f	g	h	i	j	...	q	r	s	t	u	v	w	x	y	z
doc_id																					
1	1.0	0.0	0.0	5.0	1.0	0.0	0.0	1.0	4.0	0.0	...	0.0	0.0	0.0	0.0	0.0	1.0	0.0	5.0	0.0	2.0
2	4.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	2.0	2.0	0.0	5.0	0.0	0.0	0.0	0.0	0.0	0.0
3	2.0	0.0	0.0	0.0	0.0	3.0	0.0	0.0	2.0	0.0	...	0.0	0.0	3.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	5.0	0.0	4.0	0.0	0.0	0.0	0.0	1.0	0.0	...	0.0	0.0	0.0	4.0	0.0	0.0	0.0	1.0	0.0	0.0
5	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0.0	0.0	3.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
96	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	3.0	0.0	...	0.0	0.0	0.0	0.0	4.0	5.0	0.0	0.0	0.0	0.0
97	0.0	0.0	3.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	4.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
98	0.0	0.0	0.0	5.0	0.0	0.0	4.0	3.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
99	4.0	2.0	0.0	0.0	0.0	0.0	0.0	0.0	5.0	5.0	...	0.0	0.0	2.0	0.0	0.0	3.0	0.0	0.0	3.0	0.0
100	0.0	0.0	0.0	2.0	3.0	0.0	0.0	0.0	1.0	0.0	...	0.0	0.0	0.0	0.0	0.0	3.0	5.0	4.0	1.0	0.0

100 rows × 26 columns

bow_vector



• 2. 数据展开melt() 方法

- 与透视表相对的操作是数据展开 (melt)，也叫**反透视**。数据展开是将 DataFrame 从**宽格式转换为长格式**。宽格式的数据通常每一行表示一个个体的多项属性，而长格式的数据将每一行表示为一个属性的一个值，长格式更适合绘图或逐行处理的数据分析。

- 例 将宽格式的城市和商品类别销售数据展开为长格式**

- import pandas as pd
- # 原始宽格式 DataFrame
- data = {
- '城市': ['北京', '上海', '广州', '深圳'],
- '电子产品': [500, 700, 400, 600],
- '家具': [300, 450, 320, 410]
- }
- df = pd.DataFrame(data)
- print("初始宽格式 DataFrame:")
- print(df)
- # 数据展开
- melted_df = pd.melt(df, id_vars=['城市'], var_name='商品类别', value_name='销售额')
- print("\n展开后的长格式 DataFrame:")
- print(melted_df)

运行结果:

初始宽格式 DataFrame:

	城市	电子产品	家具
0	北京	500	300
1	上海	700	450
2	广州	400	320
3	深圳	600	410

展开后的长格式 DataFrame:

	城市	商品类别	销售额
0	北京	电子产品	500
1	上海	电子产品	700
2	广州	电子产品	400
3	深圳	电子产品	600
4	北京	家具	300
5	上海	家具	450
6	广州	家具	320
7	深圳	家具	410



6. 案例：清洗与整理员工信息数据集

- 在数据分析中，原始数据通常存在**噪声**、**不完整**、**不一致**等问题，直接使用这些数据可能导致分析结果失准。因此，数据清洗与整理是数据分析的关键步骤之一。
- 清洗操作包括**处理缺失值**、**重复值**、**不一致格式**等，目的是将数据标准化、结构化，以便后续分析。
- **例：清洗与整理员工信息数据集**
 - 假设有一份员工信息数据集，其中包含姓名、年龄、职位、工资等基本信息。由于数据来源复杂，可能存在缺失值、重复数据和格式不一致等问题。



(1) 数据准备

- 首先，创建一个模拟的数据集，以展示清洗操作中的常见问题和解决方法。

- `import pandas as pd`

- `import numpy as np`

- `# 创建包含缺失值和重复数据的 DataFrame`

- `data = {`

- `'姓名': ['张伟', '李娜', '王强', '刘洋', '张伟', None, '陈敏', '赵磊', '杨杰'],`

- `'年龄': [25, np.nan, 35, 45, 25, 30, np.nan, 55, 60],`

- `'职位': ['经理', '开发', '开发工程师', '设计', '经理', '经理', '设计师', None, '开发'],`

- `'工资': [70000, 80000, np.nan, 75000, 70000, 85000, 72000, 90000, np.nan]`

- `}`

- `df = pd.DataFrame(data)`

- `print("原始数据集：")`

- `print(df)`

- 可以看到，第 2 行和第 8 行的“工资”信息缺失；第 1 行和第 6 行的“年龄”信息缺失；“职位”列中，职位的称呼不一致；第 4 行和第 0 行是重复的记录；“姓名”列中存在缺失值。

- 输出的初始数据集：

- | | 姓名 | 年龄 | 职位 | 工资 |
|--|----|----|----|----|
|--|----|----|----|----|

- | | | | | |
|---|----|------|----|---------|
| 0 | 张伟 | 25.0 | 经理 | 70000.0 |
|---|----|------|----|---------|

- | | | | | |
|---|----|-----|----|---------|
| 1 | 李娜 | NaN | 开发 | 80000.0 |
|---|----|-----|----|---------|

- | | | | | |
|---|----|------|-------|-----|
| 2 | 王强 | 35.0 | 开发工程师 | NaN |
|---|----|------|-------|-----|

- | | | | | |
|---|----|------|----|---------|
| 3 | 刘洋 | 45.0 | 设计 | 75000.0 |
|---|----|------|----|---------|

- | | | | | |
|---|----|------|----|---------|
| 4 | 张伟 | 25.0 | 经理 | 70000.0 |
|---|----|------|----|---------|

- | | | | | |
|---|------|------|----|---------|
| 5 | None | 30.0 | 经理 | 85000.0 |
|---|------|------|----|---------|

- | | | | | |
|---|----|-----|-----|---------|
| 6 | 陈敏 | NaN | 设计师 | 72000.0 |
|---|----|-----|-----|---------|

- | | | | | |
|---|----|------|-----|---------|
| 7 | 赵磊 | 55.0 | 设计师 | 90000.0 |
|---|----|------|-----|---------|

- | | | | | |
|---|----|------|----|-----|
| 8 | 杨杰 | 60.0 | 开发 | NaN |
|---|----|------|----|-----|



(2) 数据清洗与整理步骤

- 接下来分步骤清理和整理这份数据集，确保其格式统一、内容完整，并删除冗余数据。
- 1) 检测和处理缺失数据
- 首先检测数据集中的缺失值，然后对缺失的年龄和工资信息，可选择填充或删除这些行。
 - 检测缺失数据
 - # 检测缺失数据
 - `missing_data = df.isna().sum()`
 - `print("缺失数据统计:\n", missing_data)`
 - 运行结果:
 - 缺失数据统计:
 - 姓名 1
 - 年龄 2
 - 职位 1
 - 工资 2
 - `dtype: int64`

- **处理缺失数据**

- 删除姓名和工资信息缺失的行，并使用平均值填充缺失的年龄信息。
- # 删除“姓名”列中缺失值的行
- df.dropna(subset=['姓名'], inplace=True)
- # 填充年龄列的缺失值为平均值
- df['年龄'] = df['年龄'].fillna(df['年龄'].mean())
- # 删除“工资”列中缺失值的行
- df.dropna(subset=['工资'], inplace=True)
- print(df)
- 此时，数据集更新为：
- | | 姓名 | 年龄 | 职位 | 工资 |
|---|----|-------|-----|---------|
| 0 | 张伟 | 25.00 | 经理 | 70000.0 |
| 1 | 李娜 | 42.86 | 开发 | 80000.0 |
| 3 | 刘洋 | 45.00 | 设计 | 75000.0 |
| 4 | 张伟 | 25.00 | 经理 | 70000.0 |
| 6 | 陈敏 | 42.86 | 设计师 | 72000.0 |
| 7 | 赵磊 | 55.00 | 设计师 | 90000.0 |



案例：清洗与整理员工信息数据集

• 2) 处理重复数据

- 使用drop_duplicates() 方法删除重复行。
- # 删除重复的行
- `df.drop_duplicates(inplace=True)`
- `print(df)`
- 处理后的数据集为：
- | | 姓名 | 年龄 | 职位 | 工资 |
|---|----|-------|-----|---------|
| 0 | 张伟 | 25.00 | 经理 | 70000.0 |
| 1 | 李娜 | 42.86 | 开发 | 80000.0 |
| 3 | 刘洋 | 45.00 | 设计 | 75000.0 |
| 6 | 陈敏 | 42.86 | 设计师 | 72000.0 |
| 7 | 赵磊 | 55.00 | 设计师 | 90000.0 |

• 3) 规范化数据格式

- 在“职位”列中，存在“经理”、“设计”、“设计师”等不同表述。为确保数据一致性，可以统一为全称。
- # 使用字典映射将职位名称统一为全称
- `position_mapping = {`
- `'经理': '经理',`
- `'设计': '设计师',`
- `'设计师': '设计师',`
- `'开发': '开发工程师',`
- `'开发工程师': '开发工程师'`
- `}`
- `df['职位'] = df['职位'].map(position_mapping)`
- `print(df)`
- 现在的职位列格式统一为全称：
- | | 姓名 | 年龄 | 职位 | 工资 |
|---|----|-------|-------|---------|
| 0 | 张伟 | 25.00 | 经理 | 70000.0 |
| 1 | 李娜 | 42.86 | 开发工程师 | 80000.0 |
| 3 | 刘洋 | 45.00 | 设计师 | 75000.0 |
| 6 | 陈敏 | 42.86 | 设计师 | 72000.0 |
| 7 | 赵磊 | 55.00 | 设计师 | 90000.0 |



• 4) 最终数据检查

- 检查清洗后的数据集，确保所有操作已成功完成。

- # 显示清洗后的数据集

- `print("清洗后的数据集:\n", df)`

- # 检查数据类型，确保正确

- `print("数据类型:\n", df.dtypes)`

- 运行结果:

- 清洗后的数据集:

- | | 姓名 | 年龄 | 职位 | 工资 |
|---|----|-------|-------|---------|
| 0 | 张伟 | 25.00 | 经理 | 70000.0 |
| 1 | 李娜 | 42.86 | 开发工程师 | 80000.0 |
| 3 | 刘洋 | 45.00 | 设计师 | 75000.0 |
| 6 | 陈敏 | 42.86 | 设计师 | 72000.0 |
| 7 | 赵磊 | 55.00 | 设计师 | 90000.0 |

- 数据类型:

- 姓名 object
- 年龄 float64
- 职位 object
- 工资 float64
- dtype: object



Pandas数据处理与分析的内容小结表

分类	方法/属性	描述
数据创建与导入	pd.DataFrame()	创建一个新的DataFrame对象
	pd.Series()	创建一个新的Series对象
	pd.read_csv()	从 CSV 文件读取数据到 DataFrame 中
	pd.read_excel()	从 Excel 文件读取数据到 DataFrame 中
	pd.read_sql()	从 SQL 数据库读取数据到 DataFrame 中
数据预览	df.head()	查看前几行数据（默认5行）
	df.tail()	查看后几行数据（默认5行）
	df.sample()	随机采样部分行
	df.shape	查看 DataFrame 的维度（行数 和列数）
	df.columns	查看所有列的名称
	df.info()	查看 DataFrame 基本信息，如 列名、非空值计数和数据类型
	df.describe()	生成描述性统计数据，适用于 数值型数据
数据选择与过滤	df['column']	选择单列数据
	df[['col1', 'col2']]	选择多列数据
	df.iloc[]	通过行号和列号选择数据（基 于位置）
	df.loc[]	通过行标签和列标签选择数据 （基于标签）
	df.query()	使用查询字符串选择数据
数据清洗	df.fillna()	填充缺失值
	df.drop()	删除行或列
	df.rename()	重命名列
	df.duplicated()	检查重复行
	df.drop_duplicates()	删除重复行
	df.replace()	替换特定值
数据转换	df.astype()	转换数据类型
	pd.to_datetime()	将数据转换为日期类型
	df.apply()	应用自定义函数到 DataFrame 的行或列上
	df.map()	对 Series 或 DataFrame 的每个元素应用指定函数
数据操作	df.sort_values()	按列或行排序数据
	df.sort_index()	按索引排序数据
	df.groupby()	对数据进行分组操作
	df.pivot_table()	创建透视表
	df.merge()	合并两个 DataFrame（类似 SQL 中的 JOIN 操作）
	df.join()	基于索引合并两个 DataFrame
	pd.concat()	沿指定轴连接多个 DataFrame
	df.append()	向 DataFrame 添加行
数据统计与计算	df.mean()	计算均值
	df.sum()	计算总和
	df.min()	计算最小值
	df.max()	计算最大值
	df.count()	计算非 NA 值的数量
	df.median()	计算中位数
	df.corr()	计算列间的相关系数
	df.cumsum()	计算累积和
	df.cumprod()	计算累积积
导出数据	df.to_csv()	将 DataFrame 导出为 CSV 文件
	df.to_excel()	将 DataFrame 导出为 Excel 文件
	df.to_sql()	将 DataFrame 导出到 SQL 数据库
	df.to_dict()	将 DataFrame 转换为字典



作业

- 一、理论知识
 - 1. 简述 Pandas 的作用。
 - 2. 什么是 Series 和 DataFrame? 请分别解释它们的结构和应用场景。
 - 3. Pandas中如何读取和写入数据文件? 列举常见的数据文件格式, 并说明适用场景。
 - 4. 缺失数据在数据分析中常见且不可避免。Pandas 提供了哪些方法处理缺失数据?
 - 5. 请解释如何使用布尔索引、条件筛选以及 query() 方法筛选数据。
 - 6. Pandas 中有哪些常用的数据类型转换和格式化操作?
 - 7. 数据索引在数据分析中有何重要性? Pandas 中如何进行数据排序和重新索引?
 - 8. 如何使用 Pandas 进行数据的分组与聚合? 举例说明 groupby 操作在数据分析中的应用。
 - 9. 在 Pandas 中如何进行数据合并和连接? 请简述 merge、concat 和 join 方法的应用场景。



• 二、编程实践题目

- 1. 创建一个包含随机数据的 DataFrame，包含三列 A、B 和 C，生成 10 行数据。计算每列的均值，并将其作为新行添加到 DataFrame 中。
- 2. 创建一个包含多个分组数据的 DataFrame（如多个城市、多天的温度数据），使用 groupby 计算每个城市的平均温度。
- 3. 创建两个 DataFrame，分别包含学生的成绩数据和学生的班级数据，使用 merge 将这两个 DataFrame 合并为一个。



The End !